

Remote work, team velocity & long-term capability

An agent-based model of a software-development team

Miłosz Lewandowski

Code: github.com/milolew/sprint-velocity-simulation

▪ Live demo: [interactive Streamlit app](#)

1 · Goal

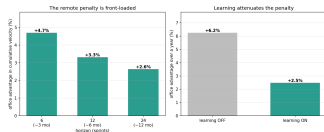
- How does the share of **remote work** affect a software team's **output** and **long-term capability**?
- **Decision support**: not one average effect, but *under which conditions* remote work is cheap vs costly.

2 · Agents & environment

- **10 developer agents**, each a per-tick state machine: idle → working → blocked → helping.
- **Environment**: a backlog of story-point tasks; time in 15-min ticks, 10-day sprints, ~1-year horizon.
- Each morning each dev is office/home w.p. `remote_share` — the one policy lever.
- **Help channel**: office ↔ office is fast/synchronous (teaches more); any-home help is slow/asynchronous.

3 · What happens · what I added

- Base model: *one sprint*, fixed skill. **My extension**: a full **year** with **learning-by-doing** & **turnover** (juniors replace leavers and must learn).
- `remote_share` now also drives **skill growth** (remote ⇒ more async help ⇒ slower ramp) and **retention**.



The office advantage shrinks over the year (left); turning learning on more than halves it (right).

4 · Key findings

- Sprint penalty scales with how often help is needed: -3.6% (skilled) to -13.2% (block-prone).
- Over a year the gap is only ~ 2.5 – 2.8% and **front-loaded, not compounding** — largest while the team is junior, then learning erases it (*reverses the prior*).
- Biggest decision-relevant effect: the **spread** of yearly output ($\sim 3\times$), not the mean.
- ⇒ **Cheap for mature, stable teams; costly for junior-heavy, high-churn ones.**